

PROJECT 1: COT 4400 ANALYSIS OF ALGORITHMS

Introduction

The project involved analyzing the time complexities of four sorting algorithms: Selection sort, Insertion sort, Merge sort, and Quick sort on personal computers. Code for each sorting algorithm was provided, requiring input to test their performance.

The inputs included:

1. Length of the array.
2. Type of sort.
3. Input type (sorted, random, or constant).

For each algorithm, the objective was to determine:

1. The minimum length of the array where the sorting algorithm completed in less than 30 milliseconds, for each input type.
2. The maximum length of the array and the corresponding time taken to sort it, for each algorithm and input type.

In essence, the project aimed to assess the efficiency of these sorting algorithms under various conditions and determine their practical performance thresholds.

Results

The results have been included in a CSV file named **Results_Project1_DiyaJain.csv**. This contains the n_{min} , t_{min} , n_{max} and t_{max} values for each type of sort performed with their respective inputs (sorted, random and constant). The table is also provided below for reference:

	nmin	tmin	nmax	tmax
SC	8500	31 ms	900000	378374 ms (6.3 mins)
SS	8500	31 ms	1000000	447494 ms (7.4 mins)
SR	8500	31 ms	1000000	445894 ms (7.4 mins)
IC	18000000	32 ms	1000000000	1741 ms (0.029 mins)
IS	17000000	31 ms	1000000000	2557 ms (0.043 mins)
IR	10010	32 ms	1000000	296049 ms (4.9 mins)
MC	800000	34 ms	1000000000	76485 ms (1.2 mins)
MS	750000	30 ms	1000000000	78118 ms (1.3 mins)
MR	260000	30 ms	1000000000	208651 ms (3.4 mins)
QC	9500	32 ms	1000000	347339 ms (5.8 mins)
QS	900000	34 ms	1000000000	54101 ms (0.9 mins)
QR	259000	30 ms	1000000000	170753 ms (2.85 mins)

Analysis

Table with ratio calculations for the algorithms is provided below:

	$t_{\max}/t_{\min}$	n ratio	$n \ln(n)$ ratio	n^2 ratio	Behavior
SC	12206	106	160	11211	n^2 (Quadratic)
SS	14435	118	180	13841	n^2 (Quadratic)
SR	14384	118	180	13841	n^2 (Quadratic)
IC	54	56	69	3086	n (Linear)
IS	82	59	73	3460	$n \ln(n)$
IR	9252	100	150	9980	n^2 (Quadratic)
MC	2250	1250	1906	1562500	$n \ln(n)$
MS	2604	1333	2043	1777778	$n \ln(n)$
MR	6955	3846	6393	14792899	$n \ln(n)$
QC	10854	105	159	11080	n^2 (Quadratic)
QS	1591	1111	1679	1234568	$n \ln(n)$
QR	5692	3861	6419	14907351	$n \ln(n)$

Analysis for each row (sorting algorithm) is as follows:

1. **SC (Selection sort with Constant array input):** For selection sort, the best, average and worst-case time complexities are all supposed to be n^2 ($\Theta(n^2)$). This complexity has been obtained in the results according to my calculations (which were obtained after running the program on my computer and noting the $n_{\min}$, $t_{\min}$, $n_{\max}$ and $t_{\max}$ values) and the code for the constant array since the ratio of $t_{\max}/t_{\min}$ is the closest to the n^2 ratio.
2. **SS (Selection sort with Sorted array input):** For selection sort, the best, average and worst-case time complexities are all supposed to be n^2 ($\Theta(n^2)$). Like in the constant array input, this complexity has been obtained in the results according to my calculations and the code for the Sorted array as the ratio of $t_{\max}/t_{\min}$ is the closest to the n^2 ratio.
3. **SR (Selection sort with Random array input):** For selection sort, the best, average and worst-case time complexities are all supposed to be n^2 ($\Theta(n^2)$). Again, this complexity has been obtained in the results according to my calculations and the code for the Random array since the $t_{\max}/t_{\min}$ is the closest to the n^2 ratio. Since the complexities for all the arrays for selection sort have lined up accurately, the computations were effective.
4. **IC (Insertion sort with Constant array input):** For insertion sort, the best-case time complexity is $\Omega(n)$. Since the array being sorted here is with constant values (so technically, the array is already sorted), the worst-case complexity was expected to be achieved. As demonstrated in the table above, this complexity ($\Omega(n)$) has been obtained when executing the code and obtaining the ratios ($t_{\max}/t_{\min}$ is closest to n ratio).

5. **IS (Insertion sort with Sorted array input):** For insertion sort, the best-case time complexity is $\Omega(n)$, the average is $\Theta(n^2)$ and best-case time complexity is $O(n^2)$. For a sorted array, insertion sort is supposed to be fast, so a time complexity of $\Omega(n)$ should be expected. However, in my calculations and execution of the code, the ratio of $t_{\max}/t_{\min}$ is the closest to the $\ln(n)$ ratio. *This is the highlighted cell (in pink) in my table.* Therefore, the expected result does not match the obtained result in this case. Possible reasons of the differing results could be:
 - a. There were other applications running on my computer (such as Google Chrome and other background processes) which could have slowed down the runtime.
 - b. The array value that I obtained for $t_{\min}$ may not have been accurate since the $t_{\min}$ itself was small (31 ms) and may have provided more accurate results for a bit larger $t_{\min}$ value (such as 33 or 34 ms).
6. **IR (Insertion sort with Random array input):** As explained before, for insertion sort, the average time complexity is $\Theta(n^2)$ and worst-case time complexity is $O(n^2)$. Insertion sort should take more time for an array with random input, so the expected complexity was $O(n^2)$, which was obtained through executing the code and performing the computations (for the ratios) in the table above.
7. **MC (Merge sort with Constant array input):** For Merge sort, the best, average and worst-case time complexities are all the same: $\Theta(\ln(n))$. Therefore, the expected complexity for running the program was $\Theta(\ln(n))$ and that was obtained through running the program and computing the ratios. It was observed that the $t_{\max}/t_{\min}$ is closest to the $\ln(n)$ ratio and therefore was accurately estimated.
8. **MS (Merge sort with Sorted array input):** The expected theoretical complexity for merge sort is $\Theta(\ln(n))$. The empirical results/ratios obtained after running the program and computing the values matched the theoretical complexity as demonstrated in the table in the first page. The obtained complexity was $\Theta(\ln(n))$, since the $t_{\max}/t_{\min}$ is closest to the $\ln(n)$ ratio.
9. **MR (Merge sort with Random array input):** As explained in the two points above, the expected theoretical complexity for merge sort is $\Theta(\ln(n))$. For the array with random values too, the empirical results obtained gave way to the complexity of $\Theta(\ln(n))$ since the $t_{\max}/t_{\min}$ was closest to the $\ln(n)$ ratio, and therefore the results were accurately obtained.
10. **QC (Quick sort with Constant array input):** For Quick sort, the best-case time complexity is $\Omega(\ln(n))$, average is $\Theta(\ln(n))$ and worst-case time complexity is $O(n^2)$. For an array with constant or same values, the expected time complexity is supposed to be the worst, so the program is supposed to run in $O(n^2)$ time. According to the results obtained by running the program and computing the ratios, the complexity is accurately estimated to be $O(n^2)$ since the $t_{\max}/t_{\min}$ was closest to the n^2 ratio. The time taken to sort this array was significantly higher than all the others and therefore explains the worstcase time complexity obtained. This array (only with quick sort) also required increasing the stack size.
11. **QS (Quick sort with Sorted array input):** As explained above, for Quick sort, the bestcase time complexity is $\Omega(\ln(n))$, average is $\Theta(\ln(n))$ and worst-case time complexity is $O(n^2)$. In this case, with an array with sorted input, it should take average time ($\ln(n)$) and it did. The results obtained clearly demonstrate that the $t_{\max}/t_{\min}$ was closest to the $\ln(n)$ ratio and therefore the desired results were obtained.
12. **QR (Quick sort with Random array input):** In this case too, an average time complexity of $\Theta(\ln(n))$ was expected since the array input consisted of random values.
The expected result was obtained, and the complexity was accurately found to be $\ln(n)$ since the ratio $t_{\max}/t_{\min}$ was closest to the $\ln(n)$ ratio after obtaining the values from running the program on my computer.